Sprawozdanie z zadania: "Grupa 4. KryptoLib z wykorzystaniem GenAI"

Autorzy:

- Daria Jaszczak (zadanie 3, sprawozdanie)
- Weronika Tyc (zadanie 1, sprawozdanie)
- Mikołaj Ziniak (zadanie 2)

Wymagania podstawowe:

1. Realizacja pełnego zakresu zadań Grup 1–3 wyłącznie z wykorzystaniem GenAI.
2. Kod oraz wszystkie jego modyfikacje muszą być tworzone za pomocą GenAI.
3. Grupa może (a nawet powinna!) analizować i testować otrzymane rozwiązania i sugerować GenAI poprawki, ale nie może samodzielnie ich wprowadzać, jeśli nie zostały podane przez GenAI.
4. Zebranie 5 najciekawszych promptów wykorzystanych podczas realizacji zadania wraz z odpowiedziami GenAI (skrótowo) i krótkim opisem, dlaczego były istotne.

Lista spełnionych wymagań: $1, 2, 3$ oraz 4.

Opis realizacji zadania:

1. Naszym zadaniem było osiągniecie tego, co robiły trzy poprzednie grupy, za pomocą ogólnodostępnych narzędzi AI. Początkowo przyjęta strategia polegała na przedłożeniu przez każdego z członków grupy promptu do każdego z zadań osobno, a następnie sklejenia wyników, z pewnymi modyfikacjami. Oczywiście całość kodu jest autorstwa GenAI. Finalnie zamieszczona wersja powstała przy użyciu Claude'a. Ponadto korzystaliśmy także z Gemini na wstępnych etapach, a także w celu weryfikacji rozwiązania zaproponowanego (sklejonego) przez Claude'a - tu Gemini nie miało zastrzeżeń.
2. Łączny czas pracy przy zadaniu oceniany jest przez poszczególnych członków zespołu odpowiednio jako 2h, 2h i 0,5h. Tak naprawdę o wiele więcej czasu niż wygenerowanie kodu i przeczytanie go zajmują jakieś poboczne sprawy (typu komunikacja w zespole czy pisanie niniejszego sprawozdania).

3. Na koniec zdecydowaliśmy się przetestować, jakie rozwiązania można uzyskać, wklejając ten sam (tylko jeden!) prompt do trzech najpopularniejszych LLMów: Claude'a, Gemini i ChataGPT. Poniżej załączony jest on wraz z odpowiedziami. Rzecz jasna pierwszy z nich poradził sobie najlepiej, drugi również nie najgorzej, zaś ostatni z jakiegoś powodu odmówił współpracy. Oprócz tego niżej znajduje się 5 najciekawszych promptów wraz z kontekstem.

   Należy dodać, że rozwiązania uzyskane w ten sposób okazały się dużo bardziej skomplikowane (a przynajmniej zawierające więcej linijek kodu) aniżeli wersja uzyskany w pierwszy sposób. Ponadto patrząc ogółem na kod wygenerowany przy pomocy AI a na ten zamieszczony na gicie przez pozostałe grupy, wydaje się on rzeczywiście nazbyt skomplikowany, mniej czytelny dla osoby nierozeznanej (choć zapewne technicznie lepszy, mniej podatny na możliwe błędy?).

Testowanie 3 różnych LLMów:

- PROMPT: WItaj. Muszę wykonać zadanie na zajęcia z kryptologii. W załączonym pliku PDF znajdują się wytyczne. Ja należę do grupy nr 4, zajmującej się rozwiązaniem za pośrednictwem GenAI właśnie. Do twojej roboty należy zatem zrobienie tego, co należało do grup 1-3. Bardzo proszę cię, abyś ostateczne rozwiązanie podał w formie pliku ZIP. Zrób to zadanie najlepiej, jak potrafisz.
  PS Pamiętaj, aby uwzględnić w przypadku podpunktu pierwszego także interpunkcję.
  (załączony do tego plik PDF z opisem zadań z eNauczania)

  – CLAUDE: Rozwiązanie poprawne, bez zastrzeżeń, natomiast różniące się od pierwotnie uzyskanej wersji (czyli wyniku wielu promptów). Claude pewne funkcje i rozwiązania zapisał w inny - zdaje się - bardziej skomplikowany sposób.

  – GEMINI: Rozwiązanie wydaje się poprawne. Co ciekawe, Gemini zdecydowała się wygenerować także sprawozdanie zawierające 5 zupełnie fikcyjnych promptów. Ponadto Gemini całość podsumował w ten sposób: "Praca z wykorzystaniem GenAI przy realizacji projektu KryptoLib 1.0 przebiegła niezwykle sprawnie. Czas wykonania całego pakietu (wraz z testami i dokumentacją) wyniósł ok. 1.5 roboczogodziny. Największym atutem AI była natychmiastowa implementacja algorytmów matematycznych (np. rozszerzony algorytm Euklidesa) oraz czysty, modułowy styl kodu w Pythonie."

  – CHAT GPT: Choć kod wygenerował, miał on problem z zapisaniem całości do ZIPa, argumentując to w następujący sposób: "It seems like I can't do more advanced data analysis right now. Please try again later.
    Przepraszam — nie udało mi się obecnie uruchomić narzędzia potrzebnego do przygotowania i przetestowania archiwum ZIP. W związku z tym nie mogę uczciwie przekazać gotowego pliku ani twierdzić, że testy zakończyły się powodzeniem."Ponadto również kreatywność poniosła go aż po wymyślanie promptów dla siebie samego.

Najciekawsze prompty:

1. Początkowo przy preprocessingu i standaryzacji alfabetu zabrakło funkcji usuwania interpunkcji, co po prostu nie było zawarte pisemnie w PDFie
   Prompt: Dodaj do funkcji czyszczących funkcję usuwającą znaki interpunkcyjne oraz dopisz ją do funkcji `pełne_czyszczenie`.
   Odpowiedź: "Oto zaktualizowany kod modułu czyszczącego. Dodałem dedykowaną funkcję `usun_interpunkcje`, która usuwa znaki interpunkcyjne oraz inne symbole specjalne za pomocą modułu string.punctuation oraz wyrażeń regularnych. Została ona również zintegrowana z funkcją `pełne_czyszczenie`.
   Zaktualizowany kod funkcji czyszczących [...]"

2. W przypadku wdrażania kodu GADERYPOLUKI Gemini popełniło literówki, które następnie trzeba było poprawiać - błąd w zasadzie karygodny, natomiast widocznie darmowe wersje są i na takie rzeczy podatne w 2026.
   Prompt: "Znalazłem literówkę i zamiast GADERYPOLUKI jest GADERPOLUKI. Zapomniałeś o Y po R. [podany kod]"
   Odpowiedź: "Masz absolutną rację – w funkcji `klucz_gaderypoluki` wkradła się literówka GADERPOLUKI zamiast poprawnej formy GADERYPOLUKI (brakowało litery Y po R). Dodatkowo upewniłem się, że wariant drugi również używa poprawnej formy POLITYKARENU (zamiast POLITIKARENU), co zapobiega powtórzeniu litery I i gwarantuje poprawną bijekcję (unikalny 26-znakowy alfabet).
   Oto poprawiony i gotowy do wklejenia kod: [...]"

3. Przy pierwotnej konwencji, jako że nie wszystko zostało wygenerowane za jednym razem i należało to sklejać, Claude wysunął kilka trafnych propozycji i uwag dotyczących preprocessingu.
   Prompt: Pomińmy
   Odpowiedź: "[...] Założenia, które warto uzgodnić z resztą grupy:
   Kolejność ALFA37: A–Z, potem 0–9, na końcu spacja, zgodnie z kolejnością z treści zadania. Jeśli osoba od preprocessingu przyjęła inną, zmień tylko stałą ALFA37.
   Cezar i Atbasz w ALFA37: działają na całym 37-znakowym alfabecie. Cezar przesuwa modulo 37, a Atbasz odwraca cały alfabet (np. A-spacja).
   ROT13 i Gaderypoluki w ALFA37: zmieniają tylko litery, a cyfry i spacja zostają bez zmian. Dzięki temu pozostają inwolucjami, czyli szyfrowanie i deszyfrowanie to to samo.
   Znaki spoza alfabetu: encrypt i decrypt rzucają ValueError. Zakładam, że tekst jest wcześniej oczyszczony modułem z podpunktu 1, więc nic nie jest cicho pomijane.
   Możesz mi wysłać wyniki tej części od kolegi z grupy, a ja sprawdzę, czy format wejścia pasuje do encrypt. Klucze Cezara, Atbasz i ROT13 robi też osoba od podpunktu 2, więc dobrze będzie porównać wasze klucze, bo Grupa 5 i tak będzie to zestawiać. [...]"

4. Po wysłaniu części pierwszej taką otrzymałam odpowiedź:
   "Sprawdziłem, jak współpracują oba moduły. Dane z `tryb_alfa26` i `tryb_alfa37` przechodzą bez problemu przez encrypt i decrypt dla wszystkich czterech szyfrów (Cezar, Atbasz, ROT13, Gaderypoluki) w obu trybach, a deszyfrowanie zwraca oryginał. Alfabety obu części są zgodne: A–Z, cyfry i spacja, więc nic nie trzeba zmieniać po żadnej stronie.
   Przy okazji znalazłem trzy drobiazgi w części pierwszej. Żaden nie psuje współpracy, ale warto je zgłosić koledze albo opisać w sprawozdaniu:

Spacje na brzegach w ALFA37. Wejście "Ala ma kota "daje ' ALA MA KOTA ', czyli ze spacją na początku i na końcu. Jeśli to niepożądane, wystarczy dodać .strip(). To kwestia ustalenia, czy spacja brzegowa jest "legalnym" znakiem tekstu jawnego.
Podwójna spacja po usunięciu znaków. W przykładzie wyszło LODZ SS, bo ø zostało usunięte po zamianie białych znaków na pojedyncze spacje. Rozwiązaniem jest przeniesienie re.sub(r'\s+', ' ', ...) na koniec, po filtrze `[^A-Z0-9 ]`.
Znaki bez rozkładu NFD. ø i đ znikają zamiast stać się o i d, podobnie jak ł, którą kolega obsłużył ręcznie. Dla polskiego tekstu to bez znaczenia, ale ß → SS działa tylko przypadkiem, przez .upper().
Cieszę się, że moduł się przydał. Do sprawozdania Grupy 4 może się przydać opis tego testu integracyjnego i tych znalezionych drobiazgów, bo to konkretny przykład weryfikacji pracy zespołu."

5. Dla pewności zdecydowałam się także poprosić Claude'a o dodatkowe testy demonstracyjne, tak aby dla zwykłego człowieka sposób działania programu był bardziej widoczny. Ostatecznie i tak natomiast walidacja takowego demo została zlecona drugiemu AIowi z racji na żmudność owych testów.
Prompt: "Czy mogłabym prosić o szerszą demonstrację możliwości tego programu? Chcę, aby wszystkie możliwe funckje zostały przetestowane, najlepiej na niebanalnych przykładach."
Odpowiedź: "Przygotowałam... a raczej przygotowałem `demo_rozszerzony.py`. Uruchomiłem go u siebie i zakończył się wynikiem 90 sprawdzeń OK, 0 błędów.
Jak uruchomić: [...]
Co jest przetestowane (każda funkcja modułu):
Preprocessing + szyfr: trudny polski tekst („Zażółć gęślą jaźń... 2026... PG-4711 (Łódź → Źródło)") przechodzi przez `try_alfa26` i `tryb_alfa37` z części kolegi, a potem przez szyfry.
Alfabety i `get_alphabet`: długości, kolejność, nieznany tryb.
encrypt / decrypt: losowe klucze i klucze budowane z hasła (KRYPTOGRAFIA), sprawdzenie monoalfabetyczności, pusty tekst, zły klucz.
`invert_key`: odwrotność odwrotności oraz równoważność szyfrowania kluczem odwrotnym i deszyfrowania.
`validate_key`: 8 rodzajów błędnych kluczy z wypisanymi komunikatami.
Znaki spoza alfabetu: małe litery, cyfry w ALFA26, diakrytyki, interpunkcja.
Cezar: przesunięcia od -1000 do 1000, złożenie dwóch przesunięć, klasyczny VENI VIDI VICI, zachowanie cyfr i spacji w ALFA37.
Atbasz, ROT13: inwolucje, różnica ROT13 i Cezara +13 w ALFA37.
Gaderypoluki: trzy frazy, własna fraza, błędne frazy.
Szyfry szczególne kontra szyfr ogólny: wynik jest identyczny.
Obciążenie: 200 000 znaków i 200 losowych round-tripów.
Łamanie Cezara brute force: program sam wskazuje przesunięcie 17 na podstawie częstości liter.
Cały potok: plik → preprocessing → szyfr → kolumny po 5 liter → zapis → wczytanie → deszyfrowanie.
Dwie uwagi na koniec. Demo utworzy w folderze skryptu pliki `demo_jawny.txt` i `demo_zaszyfrowany.tx`
możesz je potem usunąć. Przy okazji wyszło też, że `tryb_alfa37` zostawia podwójną spację
w miejscu usuniętego znaku (LODZ ZRODLO). To ten sam drobiazg nr 2, który opisywałem
wcześniej, i warto go zgłosić koledze albo opisać w sprawozdaniu."

Do wygenerowania kodu zostało użyte GenAI.

Wnioski:

- Rozwiązanie uzyskane metodą mniejszych poleceń okazały się bardziej przejrzyste i mniej skomplikowane od tych tworzonych od razu w całości.
- Zatem Claude bardzo ładnie programuje, natomiast myli się co do swojej płci.